

**Asignación [2]: “CRUD/ORM nativo basado en POO y estructuras de datos
en MySQL Sakila”.**

Por Equipo Mccarthy:
Francisco Pascual
Oscarina Brazobán
Marlenis Concepción
Rainer Martínez

Asignación Presentada a la
Escuela de Informática, Facultad de Ciencias, Como Cumplimiento del Curso “Ciencia de Datos
1” (INF- 8237-03 C2),
Dr. Silverio Del Orbe Abad.

Contenido

Resumen.....	II
Introducción.....	1
Marco de referencia.....	1
Descripción del caso práctico.....	2
Arquitectura del sistema.....	2
Implementación del CRUD/ORM.....	2
Pruebas y resultados.....	3
Conclusiones.....	3
Referencias.....	5
Anexos.....	6

Resumen

Este informe presenta el desarrollo del Caso práctico 2 de la Unidad 2 de Ciencia de Datos I, consistente en la creación de un CRUD/ORM nativo en Python para operar sobre la base de datos Sakila de MySQL. La solución utiliza programación orientada a objetos para representar entidades relacionales como países, ciudades, películas e inventario mediante clases de dominio, repositorios, controladores, servicios y un contexto de base de datos. El proyecto incorpora estructuras de datos como listas de entidades, diccionarios para caché y un historial de operaciones para evidenciar el uso de abstracciones propias de Python. Para facilitar la ejecución por los integrantes del equipo, se prepararon scripts de inicio para Mac, Linux y Windows que levantan MySQL en Docker, importan Sakila, verifican la conexión y abren el menú interactivo del CRUD. Además del flujo crear, leer, actualizar y eliminar, el trabajo incluye importación y exportación en CSV y JSON, diez consultas SQL, restricciones de integridad mediante unique constraints y métricas descriptivas fundamentales como media, rango, desviación estándar, varianza y covarianza. Los resultados demuestran una arquitectura modular y escalable, cercana al patrón ORM, donde los cambios del modelo pueden mantenerse separados de la conexión, la lógica de aplicación y la interfaz de consola. La propuesta fortalece la comprensión práctica de bases de datos relacionales, consumo programático de datos y organización de código reutilizable para proyectos de ciencia de datos.

Palabras clave: CRUD, ORM, Sakila, MySQL, Python, estructuras de datos.

Introducción

La manipulación de datos en ciencia de datos requiere comprender tanto la estructura de los datos como los mecanismos usados para consultarlos, transformarlos y validarlos. En este caso práctico se utiliza Sakila, una base de datos de ejemplo de MySQL, para construir una solución que conecta Python con un sistema gestor relacional y permite ejecutar operaciones CRUD desde una interfaz de consola. La actividad responde al objetivo de crear un CRUD/ORM nativo basado en programación orientada a objetos y estructuras de datos. El enfoque ORM permite representar tablas como objetos de dominio y encapsular las operaciones SQL en repositorios y servicios, reduciendo la dependencia directa entre la interfaz de usuario y las consultas de base de datos (Fowler, 2002).

Marco de referencia

Un CRUD agrupa las operaciones crear, leer, actualizar y eliminar, necesarias para administrar registros persistentes. En bases de datos relacionales, estas acciones se implementan mediante instrucciones SQL como INSERT, SELECT, UPDATE y DELETE. Oracle (s.f.) presenta Sakila como una base de datos de ejemplo útil para practicar relaciones entre tablas de un sistema de alquiler de películas. Python facilita este tipo de solución por su soporte para clases, dataclasses, listas, diccionarios y módulos reutilizables. La documentación oficial de Python describe las clases como un mecanismo para combinar datos y comportamiento dentro de una misma estructura (Python Software Foundation, s.f.). Docker, por su parte, permite ejecutar servicios como MySQL en contenedores reproducibles, lo que reduce diferencias entre equipos de trabajo (Docker, s.f.).

Descripción del caso práctico

El proyecto gestiona las entidades country, city, film e inventory de Sakila. Estas entidades cubren los requisitos de países, ciudades, películas e inventario planteados en la asignación. La conexión se configura mediante variables de entorno y puede ejecutarse con MySQL local o con el contenedor Docker preparado para el proyecto. El alcance técnico incluye diez consultas SQL, operaciones CRUD, importación y exportación de datos en CSV y JSON, métricas descriptivas y restricciones de integridad para mejorar la calidad de los datos. La solución se orienta a demostrar dominio de estructuras de datos, separación de responsabilidades y consumo programático de una base de datos relacional.

Arquitectura del sistema

- db.py: administra la conexión a MySQL y el manejo transaccional.
- dbcontext.py: centraliza repositorios, caché e historial de operaciones.
- models.py: define entidades y ModelCollection como lista de objetos.
- repositories.py: implementa CRUD genérico y repositorios concretos.
- controllers.py y services.py: separan el flujo de aplicación de la persistencia.
- structures.py: contiene caché de entidades e historial de consultas.
- metrics.py y reports.py: calculan media, rango, varianza, desviación y covarianza.
- import_export.py: permite importar y exportar datos CSV y JSON.
- main.py: ofrece el menú de consola para ejecutar la demostración.

Implementación del CRUD/ORM

La capa BaseRepository recibe una clase de modelo y construye consultas parametrizadas para crear, buscar por identificador, listar, actualizar y eliminar registros. Cada repositorio

concreto hereda esa funcionalidad para una entidad específica. La interfaz de consola permite seleccionar países, ciudades, películas e inventario, introducir datos y observar el resultado devuelto como objeto. La implementación evita credenciales escritas directamente en el código y usa variables de entorno. Además, los scripts quickstart de Mac, Linux y Windows simplifican la preparación de MySQL/Sakila con Docker para que el equipo pueda repetir la demostración en diferentes computadoras.

Pruebas y resultados

Las pruebas unitarias disponibles validan estructuras de datos usadas por el proyecto. En la verificación local se ejecutó pytest sobre el Caso 2 y el resultado fue de dos pruebas aprobadas. También se comprobó la conexión a MySQL Sakila por Docker, obteniendo conexión exitosa, nombre de base de datos sakila y versión MySQL 8.0.46.

- Create: creación de países de prueba desde el menú.
- Read: búsqueda por ID y listado de entidades.
- Update: modificación de registros seleccionados.
- Delete: eliminación de registros de prueba.
- Métricas: cálculo descriptivo sobre longitud, tarifa y costo de películas.
- SQL: ejecución de diez consultas y script de unique constraints.

Conclusiones

La solución cumple el objetivo principal de implementar un CRUD/ORM nativo sobre Sakila usando POO y estructuras de datos. La separación entre contexto, modelos, repositorios, controladores y servicios facilita ampliar el sistema a nuevas tablas sin reescribir toda la aplicación. La automatización con Docker mejora la reproducibilidad para los integrantes del

equipo y permite generar evidencias reales de ejecución. Como mejora futura, se recomienda ampliar las pruebas de integración, agregar validaciones de negocio más detalladas antes de insertar o actualizar datos y completar anexos con capturas finales de cada operación requerida por la rúbrica.

Referencias

marlenis-concepcion. (2026). CienciaDatosUasd2026: caso_practico_2_sakila_crud_orm

[Repositorio en GitHub]. GitHub.

[https://github.com/marlenis-concepcion/CienciaDatosUasd2026/tree/main/
caso_practico_2_sakila_crud_orm](https://github.com/marlenis-concepcion/CienciaDatosUasd2026/tree/main/caso_practico_2_sakila_crud_orm)

Docker. (s.f.). Docker documentation. <https://docs.docker.com/>

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Oracle. (s.f.). Sakila sample database. MySQL documentation.

<https://dev.mysql.com/doc/sakila/en/> Python Software Foundation. (s.f.).

The Python tutorial. <https://docs.python.org/3/tutorial/>

Anexos

- Anexo A. Evidencias de ejecución: capturas de conexión, CRUD.

Figura 1

Conexión mySQL docker y menú principal.

```
francisco@debian13:/mnt/AE0E674F0E670F9F/Master-Data-Science-AI-Portfolio/Data-Engine
Quickstart Linux - Sakila CRUD/ORM
Requisitos: Docker activo, permisos para usar docker y Python 3 instalado.

==> Verificando Docker
==> Iniciando contenedor sakila-mysql
==> Esperando a que MySQL este listo
==> Sakila ya parece estar importada (23 tablas)

==> Probando conexion desde Python
Conexion exitosa a MySQL.
Base de datos: sakila
Version MySQL: 8.0.46

==> Abriendo menu CRUD/ORM

CRUD/ORM Nativo Sakila - Mccarthy Team
1. Países
2. Ciudades
3. Peliculas
4. Inventario
5. Metricas descriptivas de peliculas
6. Historial de operaciones
0. Salir
Opcion: █
```

Figura 2*Crear país.*

```

CRUD/ORM Nativo Sakila - Mccarthy Team
1. Países
2. Ciudades
3. Peliculas
4. Inventario
5. Metricas descriptivas de peliculas
6. Historial de operaciones
0. Salir
Opcion: 1

Países
1. Crear país
2. Buscar país por ID
3. Listar países
4. Actualizar país
5. Eliminar país de prueba
Opcion: 1
País: pruebadepais
{'country_id': 111, 'country': 'pruebadepais', 'last_update': datetime.datetime(2026, 6, 5, 19, 59, 2)}

```

Figura 3*Listar países.*

```

Países
1. Crear país
2. Buscar país por ID
3. Listar países
4. Actualizar país
5. Eliminar país de prueba
Opcion: 3
Limite: 10
{'country_id': 111, 'country': 'pruebadepais', 'last_update': datetime.datetime(2026, 6, 5, 19, 59, 2)}
{'country_id': 110, 'country': 'PRUEBA PAIS', 'last_update': datetime.datetime(2026, 6, 5, 3, 18, 8)}
{'country_id': 109, 'country': 'Zambia', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 108, 'country': 'Yugoslavia', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 107, 'country': 'Yemen', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 106, 'country': 'Virgin Islands, U.S.', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 105, 'country': 'Vietnam', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 104, 'country': 'Venezuela', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 103, 'country': 'United States', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 102, 'country': 'United Kingdom', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}

```

Figura 4

Crear Ciudad.

```
Opcion: 1
Países disponibles:
{'country_id': 111, 'country': 'pruebadepais', 'last_update': datetime.datetime(2026, 6, 5, 19, 59, 2)}
{'country_id': 110, 'country': 'PRUEBA PAIS', 'last_update': datetime.datetime(2026, 6, 5, 3, 18, 8)}
{'country_id': 109, 'country': 'Zambia', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 108, 'country': 'Yugoslavia', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 107, 'country': 'Yemen', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 106, 'country': 'Virgin Islands, U.S.', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 105, 'country': 'Vietnam', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 104, 'country': 'Venezuela', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 103, 'country': 'United States', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 102, 'country': 'United Kingdom', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 101, 'country': 'United Arab Emirates', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 100, 'country': 'Ukraine', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 99, 'country': 'Tuvalu', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 98, 'country': 'Turkmenistan', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 97, 'country': 'Turkey', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 96, 'country': 'Tunisia', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 95, 'country': 'Tonga', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 94, 'country': 'Thailand', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 93, 'country': 'Tanzania', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 92, 'country': 'Taiwan', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 91, 'country': 'Switzerland', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 90, 'country': 'Sweden', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 89, 'country': 'Sudan', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 88, 'country': 'Sri Lanka', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 87, 'country': 'Spain', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 86, 'country': 'South Korea', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 85, 'country': 'South Africa', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 84, 'country': 'Slovakia', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 83, 'country': 'Senegal', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
{'country_id': 82, 'country': 'Saudi Arabia', 'last_update': datetime.datetime(2006, 2, 15, 4, 44)}
Ciudad: ciudadprueba2
country_id: 100
{'city_id': 602, 'city': 'ciudadprueba2', 'country_id': 100, 'last_update': datetime.datetime(2026, 6, 5, 20, 29, 55)}
```

Figura 5

Crear película.

```
Peliculas
1. Crear pelicula de prueba
2. Buscar pelicula por ID
3. Listar peliculas
4. Actualizar pelicula
Opcion: 1
Titulo: pruebapelicula
Descripcion: pelicularandom
Anio: 2026
language_id: 1
{'film_id': 1001, 'title': 'PRUEBAPELICULA', 'description': 'pelicularandom', 'release_year': 2026, 'language_id': 1, 'rental_duration': 3, 'rental_rate': Decimal('4.99'), 'length': None, 'replacement_cost': Decimal('19.99'), 'rating': 'G', 'last_update': datetime.datetime(2026, 6, 5, 20, 9, 47)}
```

Figura 6

Crear inventario.

```

CRUD/ORM Nativo Sakila - Mccarthy Team
1. Países
2. Ciudades
3. Películas
4. Inventario
5. Métricas descriptivas de películas
6. Historial de operaciones
0. Salir
Opcion: 4

Inventario
1. Crear inventario
2. Buscar inventario por ID
3. Listar inventario
Opcion: 1
film_id: 10
store_id: 1
{'inventory_id': 4582, 'film_id': 10, 'store_id': 1, 'last_update': datetime.datetime(2026, 6, 5, 20, 11, 10)}

```

Figura 7

Métricas películas.

```

CRUD/ORM Nativo Sakila - Mccarthy Team
1. Países
2. Ciudades
3. Películas
4. Inventario
5. Métricas descriptivas de películas
6. Historial de operaciones
0. Salir
Opcion: 5
{'length': {'mean': 115.272, 'range': 139.0, 'variance': 1632.654016, 'std_dev': 40.40611359683086}, 'rental_rate': {'mean': 2.98, 'range': 4.0, 'variance': 2.7079000000000004, 'std_dev': 1.6455698101265714}, 'replacement_cost': {'mean': 19.983999999999998, 'range': 20.0, 'variance': 36.57596399999999, 'std_dev': 6.047806544525047}, 'covariance_length_rental_rate': 1.98072, 'covariance_length_replacement_cost': 1.3696320000000002}

```

Figura 8

Eliminar país.

```
Países
1. Crear país
2. Buscar país por ID
3. Listar países
4. Actualizar país
5. Eliminar país de prueba
Opcion: 5
ID: 112
Eliminado.

CRUD/ORM Nativo Sakila - Mccarthy Team
1. Países
2. Ciudades
3. Peliculas
4. Inventario
5. Metricas descriptivas de peliculas
6. Historial de operaciones
0. Salir
Opcion: 1

Países
1. Crear país
2. Buscar país por ID
3. Listar países
4. Actualizar país
5. Eliminar país de prueba
Opcion: 2
ID: 112
No se encontro el registro.
```